FAI Final Project Report

B12902125 張晏華

1 Methods Tried

I selected a representative player from each of the methods I tested to compete against one another in order to compare their performance. The results are shown in Figure 1. The core idea of each player is summarized in the table below:

Family	Idea	Avg Rank	Outcome
`BP13` (final P2)	Determinized PIMC + opponent inference	**2.21**	**Best**; inference is the key lever
`BP16`	`BP13` + belief-floor robustness hedge	2.23	Strong + safer in unknown pools
`BP18` (final P1)	DUCT / regret-matching matrix game	2.24	Robust best-response over a fixed archetype prior
`BP26`	Opponent-as-planner archetype	2.25	Local wash vs. `BP13`; no real gain
`BP8`	Greedy / bust-probability heuristic	2.30	Fast, no opponent model; saturates
`BP29`	ISMCTS (search over my moves)	2.30	Heavier search, no opponent model; only mid-pack
`BP31`	AlphaZero self-play (search-free)	2.33	Matched `BP13` at zero cost; did not exceed it
`BP17`	Diverse rollout / lookahead	2.36	Decorrelated 2nd-entry candidate
`BP30`	AlphaZero *distillation*	2.36	Caps at teacher; search not representable
`BP19`	REINFORCE MLP policy (no search)	2.75	Fails; ties baseline B8, far below search

Key lessons. (i) My PIMC + opponent-inference agents (`BP13`/`BP16`) top the field and are the only seats clearly above the strongest baseline B10
(ii) Learned policies without test-time search are bounded by their teacher: distillation (`BP30`) saturates and pure REINFORCE (`BP19`) collapses to 2.75, level with a baseline.
(iii) Self-play AlphaZero (`BP31`) genuinely matched `BP13`-class play at *zero* inference cost (2.33), a strong result, yet still did not overtake the search agent.
(iv) Heavier test-time search is not a free win: ISMCTS (`BP29`, 2.30) spends a far larger compute budget than a cheap rollout but only lands mid-pack, confirming that *modeling opponents* is a better use of compute than searching deeper.
Choosing the final pair. I did not simply submit the two top-ranked seats since `BP13` and `BP16` are the same engine differing only by a belief-floor value. Instead, I paired the strongest inference agent (`BP13`, Agent 2) with `BP18` (Agent 1), which reaches almost identical strength through a completely different mechanism. This gives a genuinely decorrelated pair.

Final Standings (Sorted by Avg Rank):

Rank	ID	Class	Label	Avg Rank	Score	Est. Elo	Avg Score	Games	Note
1	1	BestPlayer13	-	2.21	100.00	1571	9.58	2400	
2	2	BestPlayer16	-	2.23	100.00	1569	9.61	2400	
3	4	BestPlayer18	-	2.24	100.00	1562	9.79	2400	
4	6	BestPlayer26	-	2.25	100.00	1559	9.69	2400	
5	13	Baseline10	B10	2.25	100.00	1558	9.66	2400	
6	0	BestPlayer8	-	2.30	100.00	1549	10.11	2400	
7	7	BestPlayer29	-	2.30	100.00	1550	10.08	2400	
8	9	BestPlayer31	-	2.33	100.00	1539	9.92	2400	
9	3	BestPlayer17	-	2.36	100.00	1535	10.21	2400	
10	8	BestPlayer30	-	2.36	99.84	1534	10.47	2400	
11	12	Baseline9	B9	2.56	47.40	1487	11.83	2400	
12	10	Baseline7	B7	2.60	35.13	1475	12.53	2400	
13	11	Baseline8	B8	2.75	0.00	1447	13.20	2400	
14	5	BestPlayer19	-	2.75	0.00	1445	13.82	2400	
15	15	RandomPlayer	(PAD)	3.25	-	1312	18.31	2400	(PAD)
16	14	RandomPlayer	(PAD)	3.26	-	1310	18.11	2400	(PAD)

Figure 1: Final standings of the 16-seat local round-robin (2400 games per seat): my candidate agents, TA baselines B7–B10, and two random pads.

2 Core Idea

Agent 1: Monte Carlo Sampling with Regret Matching

The core idea of Agent 1 (`BestPlayer1`) is to simplify the massive decision space using a pool of heuristic policies and apply Regret Matching to approximate a Nash Equilibrium.

- **Heuristic Policy Pool:** The algorithm incorporates five fundamental decision-making logics (e.g., avoiding busts `_bust_pick`, tight fitting `_tightfit_pick`, lookahead high card `_lookahead_hi_pick`). These heuristics efficiently and reasonably cover most tactical scenarios.

- **Rollout Simulations:** During each decision phase, the system randomly distributes unseen cards to opponents (Monte Carlo Sampling) and simulates how each heuristic policy would play out until the end of the round.
- **Regret Matching:** It cross-evaluates candidate cards from the agent's hand against the predefined policies, calculating the expected rank for each card against various opponent strategies. The algorithm then runs a fixed number of iterations (`_solve_regret`) to update the cumulative regret values for each action and accumulate the average strategy, ultimately selecting the card with the highest probability mass in this average strategy (equivalently, the lowest expected rank).

Agent 2: Opponent Belief Tracking and Depth-Limited Search

Agent 2 (`BestPlayer2`) advances the framework by introducing opponent modeling and a dynamically switching search mechanism. This allows it to adapt to opponent playstyles mid-game and perform precise game-tree evaluations in the endgame.

- **Dynamic Belief Updating:** Agent 2 records and analyzes opponents' past play histories (`history_matrix`). By calculating the consistency between actual opponent actions and the predictions of the built-in heuristic policies, the algorithm uses log-likelihood to update its belief weights for each opponent's playstyle in real-time. In subsequent simulations, the agent samples opponent behaviors based on these weights (`_sample_policy`) rather than assuming purely random play.
- **Hybrid Decision Framework:**
  - *Early to Mid-Game:* It utilizes Monte Carlo rollouts similar to Agent 1, but the opponent behavior models are calibrated using the aforementioned beliefs, making the simulations far more representative of the actual match dynamics.
  - *Endgame (Depth-Limited Search with PIMC):* When the number of remaining cards drops below a predefined threshold (e.g., 4 cards, controlled by `_SEARCH_DEPTH_LIMIT`), the state space shrinks, and Agent 2 automatically switches to an exact recursive search (`_search_min`). This exhaustive evaluation is performed on determinized opponent hands (sampled via Perfect Information Monte Carlo), rather than the full hidden information tree. It evaluates all possible game continuations within these sampled subgames and selects the optimal endgame strategy using a comprehensive scoring function that prioritizes expected rank, using expected self-penalty points (`_PEN_LAMBDA`) as a secondary tie-breaker.

3 Implementation

3.1 Hyperparameter Tuning

The behavior and efficiency of both agents are heavily parameterized. Table 1 outlines the key hyperparameters configured and investigated during development.

For Agent 1, the `_PRIOR` weights were empirically derived by profiling baseline agents. Stronger baselines favored `_bust_pick` and `_lookahead_hi_pick`, so they received the highest weights (0.30), while minority styles received monotonically decaying weights (0.20, 0.12, 0.08). To avoid overfitting, fine-tuning these parameters further was deemed unnecessary as this configuration already performed robustly. Additionally, controlled testing on the blending parameter `_ADV` confirmed that 0.5 yielded the optimal performance.

For Agent 2, the parameters `_SEARCH_DEPTH_LIMIT` and `_PEN_LAMBDA` were kept at their default values of 4 and 0.003. Setting the depth limit to 4 ensures that the recursive tree search completes safely within the execution time limit without causing any timeout violations. Meanwhile, the penalty coefficient $\lambda = 0.003$ serves its purpose by preventing the agent from making short-sighted plays that blindly avoid immediate points but ruin the overall tournament ranking. There was no further tuning attempted, as these settings proved sufficient and highly effective in practice.

Table 1: Key Hyperparameters for Agent 1 and Agent 2

Hyperparameter	Value	Description
Agent 1		
`_ADV`	0.5	Weight balancing the prior policy and regret-matching strategy.
`_RM_ITERS`	200	Iterations for the Regret Matching solver.
`_PRIOR`	[0.3, 0.3, 0.2, 0.12, 0.08]	Initial belief distribution for the 5 heuristic profiles.
Agent 2		
`_SEARCH_DEPTH_LIMIT`	4	Hand-size threshold triggering the exact endgame search.
`_PEN_LAMBDA`	0.003	Trade-off weight between expected rank and actual penalty.
`_BELIEF_FLOOR`	0.06	Min. probability in opponent models to prevent over-fitting & log(0).
`_INFER_SAMPLES`	6	Monte Carlo samples drawn to infer opponent archetypes per step.

3.2 Algorithmic and Code Optimizations

- **Action Space Pruning:** Instead of evaluating every single card in a large hand through the expensive regret matching solver, Agent 1 utilizes `_candidate_indices` to filter down the action space. By pre-selecting candidate cards recommended by the underlying heuristics, the matrix dimensionality for the regret computation is severely truncated, yielding an order-of-magnitude speedup.
- **Lightweight State Abstraction:** Full replication of the grid board state during deep rollouts is computationally prohibitive. Both agents abstract the game board into three primitive one-dimensional arrays: `row_ends` (the trailing card of each row), `row_lens` (the element counts), and `row_pens` (the pre-calculated point totals). Operations like card placement and scoring are conducted directly on these flat lists, maximizing rollout throughput.
- **Incremental Belief Tracking:** In Agent 2, evaluating opponent histories across all past rounds is optimized via an incremental processing pointer (`self._proc`). Instead of recalculating log-likelihood scores from round 0 at every turn, the belief routine only digests newly appended game matrix rows, converting an otherwise $O(T^2)$ history-parsing bottleneck into an efficient $O(1)$ amortized update.
- **Asymmetric Time Budgeting:** To prevent timeout violations while maximizing CPU utilization, both agents employ an anytime loop but with tailored budgets. Agent 1 utilizes a conservative 0.70s budget to accommodate the high execution-time variance of pure Monte Carlo rollouts. Conversely, Agent 2 is granted a larger 0.90s budget, capitalizing on the highly predictable execution time of its exact Minimax search in the endgame.

4 Self-Assessment

What I did well: The single most valuable decision was identifying opponent modeling, not raw search depth, as the dominant factor. Once I committed to determinized PIMC with Bayesian policy inference, a cheap search could route penalties onto exploitable seats. Working in strictly single-threaded pure Python, I squeezed out determinization count through precomputed board arrays, incremental $O(1)$ belief updates, and an anytime loop that safely maximizes the time budget regardless of machine load.
What I struggled with: My biggest lesson was that local proxy evaluations did not track the real platform. A classmate (b12902023) had set up a public server for everyone to upload their players and compete against each other. For instance, my ISMCTS variant looked highly competitive in my local test, but collapsed when uploaded to this public server ($\sim$216 vs. $\sim$257 for the PIMC agents). Therefore, I had to re-anchor every decision on the real-world feedback from that public platform. Furthermore, I repeatedly failed to make a learned policy surpass test-time search. Reinforce and AlphaZero distillation both saturated at their teacher's strength. Learning a value good enough to replace search proved harder than expected for this game. Finally, on shared, loaded hardware, wall-clock comparisons were unreliable, which forced me to switch to fixed-work budgets. The same noise made fine hyperparameter tuning slow, so a few design choices (e.g. the opponent prior in the regret-matching variant) remained principled hand-set values rather than fully swept optima.

5 Strategic Profiling

Agent 1: Performs well against "standard" opponents who play predictably to minimize penalties, as its `_PRIOR` distribution is tailored to mirror these conventional baselines. However, it struggles against tables dominated by highly unpredictable players or trolls, as it assigns very low probabilities to edge-case behaviors (e.g., `_low_pick`) and may fail to anticipate illogical moves.
Agent 2: Excels against players with a consistent playstyle by tracking histories and dynamically updating log-likelihood beliefs to heavily exploit them. Its endgame exact search also punishes opponents lacking deep calculation. Conversely, it is vulnerable to highly deceptive opponents who drastically switch styles mid-game, as the cumulative historical data will cause its belief weights to lag, leading to mispredictions.

6 Agent Comparison

The trade-offs between the two agents can be summarized across three dimensions:

- **Adaptability:** Agent 1 prioritizes robustness, using a fixed prior to approximate a Nash Equilibrium, making it highly resilient against unknown or strong opponents. Agent 2 sacrifices baseline robustness for targeted exploitation, dynamically adapting to predictable players but risking overfitting if opponents suddenly shift styles.
- **Resource Allocation:** Agent 1 maximizes computational efficiency through wide Monte Carlo rollouts (breadth). Agent 2 allocates its budget towards deep inference and exact endgame PIMC search (depth).
- **Risk Tolerance:** Agent 1 strictly optimizes for expected rank, making it willing to accept high-penalty gambles if it improves final placement. Agent 2 is more risk-averse; by incorporating the `_PEN_LAMBDA` weight, it balances rank optimization with damage control.